

Prix au m² dans l'Hérault

Analyse des mutations immobilières DVF 2024

Back-end de WebApp adossé à MongoDB — Projet final

Tahina HONI-RIKA · Houda OUADAH

IPSSI Montpellier — Mastère Dév, Data & IA, 4^e année

Module MIA4 — Conception et intégration d'un SGBD NoSQL

Dépôt Git : https://github.com/HoniTahina/Projet_final_mongo.git

i) Problématique

Contexte métier

Un service de la DSI (ou une agence immobilière régionale) souhaite disposer d'un outil interne pour objectiver le niveau des prix immobiliers dans l'Hérault, commune par commune, à partir de données de transactions réelles. Aujourd'hui, ce type d'analyse est fait manuellement, commune par commune, sans vue d'ensemble ni recherche géographique. L'application développée ici répond à ce besoin : un back-end interrogeable par une future application web ou mobile, exposant des indicateurs de marché calculés à la demande.

Questions métier

Les trois questions suivantes ont été formulées avant l'écriture du moindre pipeline, telles qu'un décideur les poserait :

- Quelle est l'évolution du prix au m² pour une commune donnée, année après année ?
- Quelles sont les 10 communes les plus chères de l'Hérault, en prix au m² moyen ?
- Quelles mutations immobilières ont eu lieu dans un rayon donné autour d'un point géographique (ex. : autour d'une adresse recherchée) ?

Jeu de données

Source : Demandes de Valeurs Foncières (DVF), publiées par l'administration fiscale et diffusées sur data.gouv.fr, sous licence ouverte (Etalab).

```
https://files.data.gouv.fr/geo-dvf/latest/csv/2024/departements/34.csv.gz
```

Volume : 69 651 lignes brutes importées (une ligne = un lot ou une parcelle), regroupées en 29 519 mutations exploitables (cf. chapitre ii). Période couverte : année 2024, département de l'Hérault (34) uniquement — un seul département et une seule année, ce qui limite la portée de l'analyse temporelle (cf. réserves, chapitre iv).

Ce jeu convient particulièrement à nos trois questions : il contient nativement les coordonnées GeoJSON de chaque mutation (question 3), une date exploitable pour une évolution temporelle (question 1), et une clé de regroupement par commune (question 2) — au prix d'un travail de nettoyage détaillé au chapitre suivant.

Pourquoi MongoDB plutôt qu'un SGBD relationnel

Trois caractéristiques de la donnée justifient ici le choix du modèle document, indépendamment du fait que le module portait sur MongoDB :

- Structure variable par mutation : une mutation porte de 1 à plusieurs lots (jusqu'à 5 dans le CSV source), un nombre variable et non plafonné a priori. En relationnel, cela imposerait une table lots séparée avec une jointure systématique pour reconstituer une mutation ; en document, chaque mutation s'écrit naturellement comme un seul objet avec un tableau de lots imbriqué, lu et affiché en une seule requête.
- Champs très souvent absents ou vides (surface, nombre de pièces, coordonnées) : un schéma relationnel strict imposerait des colonnes NULL partout ; le modèle document absorbe nativement cette variabilité sans redéfinir de schéma à chaque nouveau cas rencontré.

- Besoin géospatial natif : la question 3 (recherche par rayon) s'appuie directement sur un index 2dsphere et l'opérateur \$geoNear, une fonctionnalité intégrée à MongoDB sans extension complémentaire, alors qu'elle nécessiterait une extension dédiée (PostGIS) côté relationnel.

ii) Problèmes de données rencontrés

Aucun jeu de données réel n'est propre. Cinq anomalies distinctes ont été détectées et traitées, chacune vérifiée par une mesure avant tout traitement.

Anomalie	Détection	Ampleur	Traitement retenu	Justification
Une mutation s'étale sur plusieurs lignes (un lot ou une parcelle par ligne)	\$group sur id_mutation, comparé au nombre de lignes brutes	69 651 lignes brutes → 29 565 id_mutation distincts (écart de 40 086 lignes)	Regroupement (\$group + \$push) par id_mutation avant tout calcul ; lots imbriqués (embed) dans la mutation	Sans ce regroupement, valeur_fonciere est comptée une fois par ligne au lieu d'une fois par vente : tout prix moyen calculé serait faux
Valeurs foncières symboliques (quasi nulles)	countDocuments sur valeur_fonciere < seuils successifs	622 mutations (≈2,1 %) < 1000 €, dont 235 à ≤1 €	Exclusion des calculs de prix (\$match valeur_fonciere ≥ 1000)	Donations, transmissions familiales ou régularisations administratives, pas des transactions de marché ; les inclure ferait chuter artificiellement le prix moyen
Mutations sans valeur foncière exploitable	distinct(id_mutation) filtré vs total, sur lots_bruts	46 mutations (29 565 – 29 519)	Exclues du regroupement final	Aucun prix calculable pour ces mutations ; les inclure produirait des valeurs nulles ou aberrantes dans les moyennes
code_postal n'identifie pas une commune de façon unique	distinct(code_postal) comparé à distinct(code_commune) sur les données brutes	81 codes postaux distincts contre 340 codes commune (INSEE) distincts	Basculer de la clé de référence vers code_commune (identifiant administratif unique) ; code_postal conservé comme simple attribut d'affichage	Un code postal est une zone de distribution postale pouvant couvrir plusieurs communes ; l'utiliser comme clé de \$lookup aurait mélangé les statistiques de communes différentes
type_local vide sur de nombreux lots (terrains non bâtis)	countDocuments sur type_local vide ou absent	7 379 mutations sans aucun type de local renseigné sur leurs lots	Règle de priorité : Maison > Appartement > premier type non vide > "Terrain / non bâti"	L'absence de type ne signifie pas une donnée manquante par erreur, mais très souvent une parcelle de terrain nu (sans bâtiment) : un cas métier réel, pas un défaut à corriger arbitrairement

Intégrité référentielle : MongoDB n'impose aucune clé étrangère

Le référentiel communes contient les communes ayant au moins une mutation avec une surface_totale > 0 : 331 communes, contre 340 codes commune distincts dans les données brutes. Neuf communes existent donc dans mutations sans entrée correspondante dans communes. Les filtres métier sur les prix appliquent ensuite valeur_fonciere ≥ 1 000 € côté API afin d'écarter les transactions symboliques. Un \$lookup sur une mutation sans référentiel renvoie un tableau vide ; la route de détail gère explicitement ce cas avec preserveNullAndEmptyArrays.

Ce que ces anomalies changent pour l'interprétation des résultats

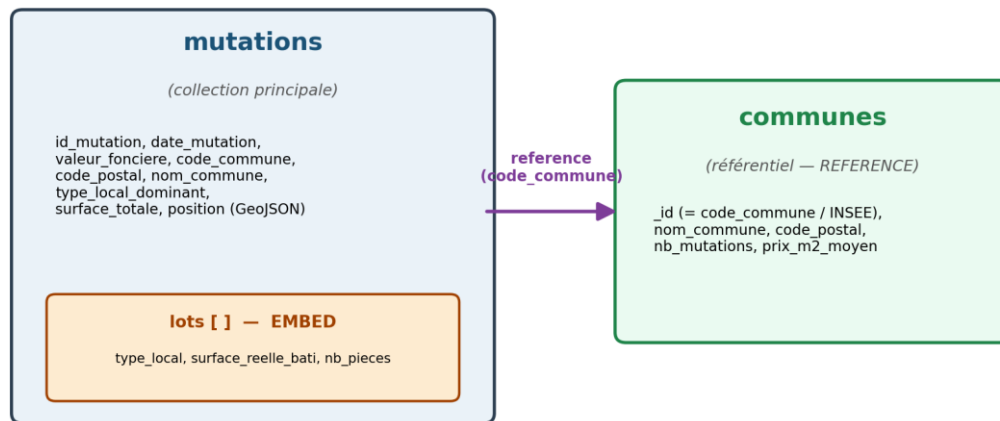
Les prix au m² présentés au chapitre iv excluent systématiquement les 622 mutations à valeur symbolique : ils reflètent des prix de marché, pas la totalité brute des actes enregistrés. La bascule vers code_commune signifie que les statistiques par « commune » ne sont plus artificiellement fusionnées entre communes limitrophes partageant un code postal — un changement de méthode

qui modifie mécaniquement le classement du chapitre iv par rapport à une première version fondée sur code_postal (erreur détectée et corrigée en cours de projet, cf. chapitre iii).

iii) Démarche

Modélisation

Deux collections liées : mutations (collection principale) et communes (référentiel léger, 331 documents après nettoyage).



Relation	Choix	Cardinalité	Taille bornée ?	Cycle de vie
Mutation → Lots	Embed	1:peu (1 à 5 lots par mutation observés)	Oui, bornée par construction (nombre de lots d'une vente)	Un lot n'existe jamais indépendamment de sa mutation
Mutation → Commune	Reference	N:1 (des milliers de mutations par commune)	Non pertinent : la commune existe indépendamment du nombre de mutations	La commune préexiste et survit à chaque mutation individuelle ; ses statistiques sont recalculées périodiquement (pattern Computed)

Ce qui nous ferait changer d'avis : nous avons initialement référencé les communes par `code_postal`, avant de découvrir par la mesure (chapitre ii) que ce champ ne les identifie pas de façon unique. Si un futur besoin métier nécessitait d'analyser par zone postale plutôt que par commune administrative, `code_postal` redeviendrait pertinent — mais comme dimension d'analyse, jamais comme clé de jointure.

Import

```
docker cp .\data\dvf34.csv projet-mongo:/tmp/dvf34.csv
docker exec projet-mongo mongoimport --username "$mongoAppUser" --password
"$mongoAppPassword" --authenticationDatabase projet \
  --db projet --collection lots_bruts --type csv --headerline --drop \
  --file /tmp/dvf34.csv
# 69 651 document(s) imported successfully. 0 document(s) failed to import.
```

Transformations appliquées ensuite par le script db/build_collections.js (aggregation pipeline, \$merge) : conversion de types (chaînes CSV → double, int, date, string), regroupement par id_mutation avec lots imbriqués, calcul et stockage de surface_totale (pattern Computed), détermination de type_local_dominant par règle de priorité, construction GeoJSON, puis construction du référentiel communes par agrégation sur les mutations valides.

Indexation

Index	Requête servie	Stage avant → après	totalDocsExamined avant → après
{ code_postal: 1, date_mutation: -1 }	GET /agg/explain?code_postal=34000 ; filtres par code_postal et plage de dates	COLLSCAN → FETCH + IXSCAN	29 519 → 1 931 (test officiel API sur code_postal 34000)
{ id_mutation: 1 } (unique)	Contrainte d'unicité (empêche la réimportation en double d'une mutation), pas une requête de lecture	—	Non mesurable par un explain de lecture : son rôle est de rejeter les doublons à l'écriture (DuplicateKeyError, géré en 409 côté API)
{ position: "2dsphere" }	GET /agg/proximite (\$geoNear)	Sans index : erreur bloquante NoQueryExecutionPlans (aucun plan d'exécution possible) → avec index : exécution normale	N/A → requête exécutée avec succès (la preuve de nécessité est ici l'échec total sans l'index, plus radicale qu'un simple ratio)

Détail de la mesure sur le premier index (capture explain() complète, chapitre iv) : ratio totalDocsExamined / nReturned de 15,3 avant l'index, ramené à 1,0 après. Le nombre de documents examinés passe de 29 519 à 1 931 pour 1 931 résultats.

Architecture applicative

Quatre tiers, conformes au schéma cible du starter fourni :

Front (HTML/JS) tableau	→ API REST (FastAPI) CRUD + /agg/* validation Pydantic	→ Driver (PyMongo) pool de connexions utilisateur applicatif	→ MongoDB collections + index
----------------------------	--	--	----------------------------------

Le front est une page HTML/JS statique servie par nginx, sans framework, appelant l'API en fetch(). L'API est développée avec FastAPI et le driver PyMongo (un seul MongoClient partagé pour tout le processus). Le conteneur MongoDB, l'API et le front sont orchestrés par un unique docker-compose.yml.

Routes exposées

Méthode	Route	Rôle
GET	/health	Statut de l'API et de la connexion MongoDB
GET	/mutations	Liste paginée, filtrable par code_commune
GET	/mutations/{id}	Détail d'une mutation, enrichi via \$lookup vers communes
POST	/mutations	Création (422 si invalide, 409 si id_mutation déjà existant)
PUT	/mutations/{id}	Modification (404 si inexistant)
DELETE	/mutations/{id}	Suppression (404 si inexistant)
GET	/agg/prix-m2-evolution/{code_commune}	Question métier 1

Méthode	Route	Rôle
GET	/agg/top10-communes	Question métier 2 (\$lookup significatif)
GET	/agg/proximite	Question métier 3 (\$geoNear)
GET	/agg/explain	Plan d'exécution de la requête la plus fréquente
POST / DELETE	/admin/index	Créer / supprimer les index (protocole de capture avant/après)

Sécurité

- Authentification MongoDB activée : aucun accès anonyme possible.
- Utilisateur applicatif app : rôle readWrite limité à la base projet. La commande `connectionStatus` a confirmé l'utilisateur authentifié app et le rôle readWrite sur projet ; l'application n'utilise donc pas le compte root pour ses opérations courantes.
- Secrets : `.env` est exclu du dépôt. La vérification `git check-ignore .env` renvoie bien `.env` et `git ls-files .env` ne retourne aucune ligne : le fichier contenant les secrets locaux n'est donc pas suivi par Git.
- Validation des entrées : modèle Pydantic strict côté API (types, bornes, longueurs) — tout champ inattendu ou hors bornes est rejeté en 422 avant d'atteindre MongoDB.
- CORS restreint à l'origine du front (`http://localhost:3000`), jamais de `allow_origins=["*"]`.

iv) Résultats

Question 1 — Évolution du prix au m² pour une commune

Question : quelle est l'évolution du prix au m² pour une commune donnée, année après année ?

```
// GET /agg/prix-m2-evolution/{code_commune}
// surface_totale est précalculée et stockée sur chaque mutation (pattern
// Computed) : pas besoin de la recalculer à chaque appel.
[
  { $match: {
    code_commune: code_commune,
    valeur_fonciere: { $gte: 1000 }, // exclut donations/régularisations
    surface_totale: { $gt: 0 },
  }},
  { $group: {
    _id: { $year: "$date_mutation" },
    valeur_totale: { $sum: "$valeur_fonciere" },
    surface_totale: { $sum: "$surface_totale" },
    n: { $sum: 1 },
  }},
  { $project: {
    _id: 0, annee: "$_id",
    prix_m2_moyen: { $round: [ { $divide: [ "$valeur_totale", "$surface_totale" ] }, 2 ] },
    n: 1,
  }},
  { $sort: { annee: 1 } },
]
```

Résultat (exemple : La Grande-Motte, code_commune 34344) :

2024 — prix moyen au m² : 4 106,69 €/m² — 714 mutations exploitables.

Interprétation :

Le jeu ne couvre qu'une seule année (2024) : la route est conçue pour supporter plusieurs années dès qu'un historique multi-annuel sera importé, mais ne renvoie aujourd'hui qu'un point par commune. Réserve à formuler explicitement à la direction : aucune tendance pluriannuelle ne peut être affirmée sur la base de ce seul jeu de données ; seul un niveau de prix pour 2024 est mesurable.

Question 2 — Top 10 des communes les plus chères

Question : quelles sont les 10 communes les plus chères de l'Hérault, en prix au m² moyen ?

```
// GET /agg/top10-communes
// $lookup significatif : mutations ne stocke que code_commune, le nom et
// le code postal d'affichage viennent du référentiel communes.
[
  { $match: { valeur_fonciere: { $gte: 1000 }, surface_totale: { $gt: 0 } } },
  { $group: {
    _id: "$code_commune",
    valeur_totale: { $sum: "$valeur_fonciere" },
    surface_totale: { $sum: "$surface_totale" },
    n: { $sum: 1 },
  }},
  { $match: { n: { $gte: 20 } } }, // écarte les communes à trop peu de ventes
  { $project: { prix_m2_moyen: { $round: [ { $divide: [ "$valeur_totale", "$surface_totale" ] }, 2 ] }, n: 1 } },
  { $sort: { prix_m2_moyen: -1 } },
  { $limit: 10 },
  { $lookup: { from: "communes", localField: "_id", foreignField: "_id", as: "commune" } },
  { $unwind: "$commune" },
  { $project: { _id: 0, code_commune: "$_id", nom_commune: "$commune.nom_commune",
```

```
code_postal: "$commune.code_postal", prix_m2_moyen: 1, n: 1 } },
]
```

Résultat :

1. Palavas-les-Flots — 4 648,52 €/m² — 242 mutations
2. Mauguio — 4 126,37 €/m² — 353 mutations
3. Bouzigues — 4 115,65 €/m² — 26 mutations
4. La Grande-Motte — 4 106,69 €/m² — 714 mutations
5. Montady — 3 856,07 €/m² — 36 mutations
6. Vic-la-Gardiole — 3 813,41 €/m² — 66 mutations
7. Clapiers — 3 723,57 €/m² — 48 mutations
8. Marseillan — 3 667,78 €/m² — 316 mutations
9. Balaruc-les-Bains — 3 383,29 €/m² — 270 mutations
10. Mudaison — 3 327,82 €/m² — 28 mutations.

Interprétation :

Le filtre $n \geq 20$ écarte les communes disposant de trop peu de mutations sur l'année, afin qu'une ou deux ventes atypiques ne suffisent pas à placer artificiellement une commune en tête. Réserve : ce classement porte sur le prix moyen, pas médian ; quelques biens très haut de gamme peuvent encore tirer la moyenne vers le haut.

Question 3 — Mutations à proximité d'un point

Question : quelles mutations ont eu lieu dans un rayon donné autour d'un point géographique ?

```
// GET /agg/proximite?lon=...&lat=...&rayon_km=...
// $geoNear DOIT être le premier stage : il exploite directement l'index
// 2dsphere de la collection source, avant toute transformation.
[
  { $geoNear: {
    near: { type: "Point", coordinates: [lon, lat] },
    distanceField: "distance_m",
    maxDistance: rayon_km * 1000,
    spherical: true,
  }},
  { $project: { _id: 0, id_mutation: 1, valeur_fonciere: 1, code_commune: 1,
    nom_commune: 1, distance_m: { $round: ["$distance_m", 0] } } },
  { $limit: 100 },
]
```

Résultat (exemple : centre de Montpellier, rayon 5 km) :

id_mutation	code_postal	valeur_fonciere	distance_m
2024-403556	34000	900 000 €	52 m
2024-406624	34000	105 500 €	52 m
2024-403341	34000	200 000 €	56 m
2024-406990	34000	700 000 €	65 m
2024-406284	34000	135 000 €	70 m

Interprétation :

Les résultats sont bien triés par distance croissante, conformément au comportement attendu de \$geoNear. Réserve : une des mutations retournées lors de nos premiers tests portait une valeur_fonciere de 1 € — une donation ou régularisation (cf. chapitre ii), à filtrer manuellement par l'utilisateur final de cette route si l'objectif est une estimation de marché plutôt qu'un inventaire brut des actes.

Capture explain() avant/après (requête la plus fréquente)

Métrique	Avant (COLLSCAN)	Après (FETCH → IXSCAN)
Stage	COLLSCAN	FETCH → IXSCAN
totalDocsExamined	29 519	1 931
totalKeysExamined	0	1 931
nReturned	1 931	1 931
Ratio examinés / rendus	15,3	1,0
Temps d'exécution	161 ms	5 ms

Commentaire : avec AUTO_INDEX=false, la requête GET /agg/explain?code_postal=34000 effectuée un COLLSCAN et examine 29 519 documents pour 1 931 résultats (ratio 15,3). Après POST /admin/index, l'index composé { code_postal: 1, date_mutation: -1 } est utilisé : le plan devient FETCH → IXSCAN, 1 931 documents sont examinés pour 1 931 résultats (ratio 1,0), et le temps observé passe de 161 ms à 5 ms.

Capture du front en fonctionnement

Immobilier Hérault — Demandes de Valeurs Foncières 2024

Source : data.gouv.fr (DVF). Base MongoDB — collections mutations (29 519 documents) et communes (161 documents).

1. Évolution du prix au m² pour une commune

Code postal :

Année	Prix moyen au m²	Nb mutations
2024	3 541 €/m²	512

2. Top 10 des communes les plus chères

Commune	Code postal	Prix moyen au m²	Nb mutations
Palavas-les-Flots	34250	4 649 €/m²	242
La Grande-Motte	34280	4 107 €/m²	714
Marseillan	34340	3 668 €/m²	316
Villeneuve-lès-Maguelone	34750	3 590 €/m²	72
Mauguio	34130	3 541 €/m²	512
Jacou	34830	3 520 €/m²	121
Montpellier	34090	3 457 €/m²	834
Balaruc-les-Bains	34540	3 302 €/m²	293
Montpellier	34000	3 261 €/m²	1612

Note : la capture du front a été réalisée pendant la phase d'intégration. Les chiffres de référence retenus dans ce rapport sont ceux des contrôles finaux effectués après reconstruction des collections (29 519 mutations exploitables et 331 communes).

v) Conclusion

Ce qui a été livré

Une API REST complète (CRUD + 3 routes d'agrégation métier) adossée à deux collections MongoDB liées, un front minimal fonctionnel, des index justifiés par des captures explain() avant/après, une sécurité de base vérifiée (authentification, utilisateur applicatif à privilèges limités, secrets hors dépôt), et une gestion d'erreurs HTTP testée (422, 404, 409, 201) — y compris un bug réel identifié en cours de développement (doublon d'id_mutation renvoyant un 500 générique) et corrigé en 409 explicite.

Ce qui n'a pas pu être fait

- Historique multi-annuel : le jeu ne couvre que l'année 2024, la route d'évolution temporelle ne peut donc pas encore démontrer de tendance réelle.
- Carte interactive (Leaflet) pour la question géospatiale : le résultat est actuellement affiché en tableau, pas en carte.

Points bonus non réalisés faute de temps : Replica Set, validateur \$jsonSchema et requête couverte. En revanche, le projet comprend bien un index géospatial 2dsphere, une validation stricte des entrées avec Pydantic, un utilisateur MongoDB applicatif à privilèges limités et une mesure explain() avant/après.

Passage à l'échelle : et à 10 millions de documents ?

À ce volume, une seule machine MongoDB deviendrait un goulot d'étranglement en écriture comme en lecture pour les requêtes larges (top 10 national, par exemple). La shard key naturelle serait `code_commune` : c'est le champ sur lequel porte la quasi-totalité des requêtes de filtrage (question 1, listing paginé), ce qui garantirait des requêtes targeted (un seul shard interrogé) plutôt que broadcast.

Limite prévisible de ce choix, chiffrée sur nos propres données : le Hérault compte 340 communes très inégalement peuplées en mutations (Montpellier concentre à elle seule plusieurs milliers de mutations quand une petite commune rurale n'en compte que quelques dizaines) — un déséquilibre analogue à celui observé au Jour 4 du cours sur le sharding par state (Texas/New York surreprésentés). Une shard key sur `code_commune` risquerait donc un chunk "chaud" pour les grandes villes. Resteraient targeted : toutes les requêtes filtrant par `code_commune` (questions 1 et 2, listing). Deviendraient broadcast : toute requête sans `code_commune`, en particulier la recherche géospatiale par rayon (question 3), qui devrait alors interroger tous les shards — un point à surveiller si le passage à l'échelle réel devait avoir lieu.

Limites de ce travail

Les données ne couvrent qu'un département et une seule année : aucune conclusion sur une tendance de marché ni sur une comparaison inter-régionale n'est possible à partir de ce seul jeu. Le prix au m² moyen reste sensible aux biens atypiques (biens de prestige, terrains agricoles classés en "Terrain / non bâti") qu'un prix médian par commune permettrait de mieux neutraliser — piste évoquée mais non mise en œuvre ici faute de temps.

Ce que nous ferions différemment

Nous vérifierions la fiabilité de la clé de référence (`code_postal` vs `code_commune`) dès la phase de cadrage plutôt qu'après une première implémentation. La correction vers `code_commune` a nécessité de reconstruire le référentiel communes et de retester les agrégations. Pour les performances, nous avons ensuite standardisé une seule démonstration reproductible via l'API : `AUTO_INDEX=false`, capture COLLSCAN, POST /admin/index, puis capture FETCH → IXSCAN.

Annexes

Schéma détaillé des collections

Voir chapitre iii — schéma des deux collections mutations / communes.

Liste complète des routes

Voir chapitre iii — tableau des routes exposées.

Commandes d'import

```
docker cp .\data\dvf34.csv projet-mongo:/tmp/dvf34.csv
docker exec projet-mongo mongoimport --username "$mongoAppUser" --password
"$mongoAppPassword" --authenticationDatabase projet \
  --db projet --collection lots_bruts --type csv --headerline --drop \
  --file /tmp/dvf34.csv
docker cp .\db\build_collections.js projet-mongo:/tmp/build_collections.js
docker exec projet-mongo mongosh --username "$mongoAppUser" --password "$mongoAppPassword"
--authenticationDatabase projet projet \
  --file /tmp/build_collections.js
```

Répartition du travail dans le binôme

Tâche	Responsable
Import et nettoyage des données, construction des collections	Houda OUADAH
Détection et traitement des anomalies	Houda OUADAH
Index et captures explain()	Houda OUADAH
Sécurité MongoDB	Houda OUADAH
API FastAPI (CRUD, agrégations, gestion des erreurs)	Tahina HONI-RIKA
Front HTML/JS	Tahina HONI-RIKA
Rapport	Les deux membres